

ENSE707: Software Quality Assurance Project

GymQ: Quality Assurance Improvements for Gym Equipment Manager and Maintenance Reporting.

**Lorenz Soriano [18011129]
Jayden Marsh [23217931]
Christian Cantos [23188023]**

GitHub repository: <https://github.com/lanieeee36/GymQ-Software-Quality-Assurance-Project>

August 2026

Problem Definition and Approval

Background and Real World Problem:

Based on our personal experience, gym-goers have no way to formally queue for equipment/machines or report maintenance issues other than verbally telling staff members their problems. This results in conflicts between gym-goers or broken equipment staying broken for longer than it should because nobody has a reliable way to report it.

Both these problems are common in most commercial and university gyms that rely on shared equipment and active staff on site. The practical relevance is high as these are everyday problems for a large portion of regular gym users.

Stakeholder Involvement

Gym-goers (all genders, and typically ages 14+):

Gym-goers can join queues for equipment/machines that are busy to have a fair way for everyone to use them. They can also report broken equipment through the app to have a reliable way to notify staff of the issues. Users can also see the operational and availability status of equipment/machines: "Available", "In use", "Out of Order", "In Progress", "Operational".

Gym Staff:

Staff currently rely on gym-goers to verbally tell them about broken equipment, which is inconsistent, unreliable, and often delayed until multiple people have talked to staff about it. With the app, staff will receive reports with information like the time, which machine, how urgent, and what the issue is. They can track and manage all the reports and flag them as "in progress" or "operational", which will update machine status for users to see.

Gym Management:

Management could benefit from usage and maintenance data, eg., which machines are the most popular, which break the most, and gym capacity/peak time patterns. This would allow them to replace the least used machines, or install more of the popular/overcrowded machines.

Testers/Developers:

The testers/developers will have to deal with a concurrent state (the queue) along with user-written, subjective inputs from the broken equipment reports. They need to make sure the queue logic stays correct with multiple users accessing at the same time, whether equipment status updates are the same across all users, and whether the system is reliable at the edges, like conflicting reports, peak loads, and race conditions.

Businesses/Gyms:

A large majority of gyms have no digital queue or reporting system and operate in the same way, with only a few staff on site during work hours and often no staff outside of work hours (gyms open 24/7). The gyms might be losing customers and money due to the issues of

overcrowded machines or broken equipment, meaning they could benefit from a digital management system for queues and equipment.

Problem Statement

Gyms rely on shared, high-demand equipment where members mostly share equipment through verbal agreements to determine whose turn it is, an approach that is inconsistent and often leads to confusion or conflict, especially during peak hours. Similarly, gym staff have no reliable way of reporting faulty equipment apart from being informed verbally by gym members, meaning there is no record of what's broken, when it was reported, or whether it has been fixed. So staff typically only become aware once a member complains or attempts to use broken equipment. What members need is a fair way to know when equipment is available and whether it's their turn to use it, while staff need a reliable way to flag equipment as faulty and track whether it requires maintenance or repair. Left unaddressed, this results in wasted time, frustration, and potential safety risks for customers and the gym business, less customer satisfaction, inconsistent equipment uptime, and no data to inform maintenance planning or identify recurring issues. A structured digital system addressing both problems would directly improve the customer experience while giving management better operational tools.

Identifying Key Software Quality Issues:

Functional Correctness

- Does the queue position calculation stay accurate as members join and leave, including when someone in the middle of the queue is removed?
- Does confirming a fault report actually remove the equipment from availability for new queue joins and sessions?

Race Conditions / Concurrency

- If two members attempt to join the same equipment's queue at the same moment, does the system assign consistent, non-conflicting positions?
- Can two end-triggers for the same session fire close together without corrupting session state?

Data Consistency / real-time accuracy

- How would the system handle conflicting fault reports?
- Is there a risk of sync delay between a member's device and the shared queue order, such that different members briefly see different queue states?

Usability

- Can a member submit a fault report in only a few taps, realistically usable mid-workout?

- Is queue position and estimated wait time displayed clearly enough to reduce members physically crowding around equipment, which is the core behaviour the system is meant to replace?

Performance

- Can the system handle peak-load conditions such as many members joining, updating, and viewing queues concurrently without degraded response time?
- Is response time fast enough to prevent two members both believing it's their turn at the same time?

Suitability for a Software Quality Assurance Project

GymQ is well suited to an SQA project because it involves clearly defined, testable state transitions such as queue join/leave, and from fault reporting to triage to resolve to close rather than purely subjective or open-ended functionality. This makes it possible to define concrete acceptance criteria, design meaningful test cases, and build a requirements traceability matrix, which are all core SQA deliverables. The system also has genuine quality risks such as data integrity, concurrency, and access control that provides substantive material for test strategy and defect analysis, rather than being a trivial CRUD exercise.

Feasibility within one semester

This project's scope is achievable within a single semester by deliberately focusing not on building a full gym management system but rather limiting it to two functional modules such as the gym queueing system and fault reporting. The project focuses on a small, well-defined set of features that can realistically be designed, implemented, and tested during a specific timeframe.

Several factors that make this project feasible are that:

- **Bounded Scope:** The prototype will not be tested or deployed in an actual gym centre. Instead, it is developed in a standalone fixed set of commonly used equipment. In this approach we reduce added complexity of actual site deployment, or real-world hardware integration that could extend the development of this project.
- **Reuse of familiar technologies:** The prototype will be built and developed in a tools and already languages that we are familiar with which is C#/.NET. By using our experiences through established coursework we can reduce the learning curve and can focus on ensuring the prototype's quality.
- **Focused quality attributes:** Rather than addressing all ISO/IEC 25010 characteristics in depth, the project prioritises four primary characteristics: Reliability, Security, Usability, Maintainability. With the remainder acknowledged but treated as

secondary or out of scope. This keeps requirements analysis, test design, and traceability work manageable.

- **Realistic prototype expectations:** The deliverable is an initial prototype demonstrating core functionality such as queue join/leave, fault report submission and status tracking rather than a production-ready system, consistent with the assignment's Task 3 expectations.

Requirement and Quality Analysis

Functional Requirements

Here are the initial identified functional requirements:

ID	Functional Requirement
FR-001	System shall allow users to join a virtual queue for a specific gym equipment
FR-002	System shall notify the next user in queue when a gym equipment becomes available
FR-003	The system shall allow a member who is next in queue to send an in-app 'nudge' notification to the current equipment user, asking them to confirm whether they are still using the equipment.
FR-004	System shall remove a user from queue if they don't claim equipment within a set of time
FR-005	The system shall allow a gym member to flag/report fault equipment.
FR-006	Staff shall be able to submit a maintenance report for malfunctioning equipment
FR-007	System shall mark equipment as "unavailable" once a maintenance report is filed
FR-008	System shall track equipment usage duration per session
FR-009	Admin shall be able to view real-time status of all gym equipments

Non-Functional Requirements

Here are the initial identified non-functional requirements:

ID	Non-Functional Requirement	Quality Attribute
NFR-001	Queue status shall update	Performance

	within 2 seconds under normal load	
NFR-002	System shall retain usage/maintenance logs for at least 6 months	Reliability
NFR-003	Only authorized staff accounts can submit maintenance report	Security
NFR-004	System shall be usable by first-time users without training	Usability
NFR--005	Mobile interface shall work on both Android and iOS devices	Compatibility
NFR-006	System shall be available 99% of the time during gym operating hours	Reliability

Requirement Quality Analysis

The initial requirements were reviewed against six requirement quality criteria: clarity, completeness, consistency, correctness, feasibility, and testability.

FR-001- The system shall allow users to join a virtual queue for a specific gym equipment.

Criterion	Assessment	Finding
Clarity	Need Improvement	The main function is understandable, but the type of user and meaning of a selected piece of equipment should be clearly defined.
Completeness	Need Improvement	Conditions for joining and the expected result after joining are not specified.
Consistency	Review Required	Must be checked against related queue requirements such as FR-002 and FR-003
Correctness	Verification Required	The function supports the proposed queueing problem but should be confirmed against stakeholder

		expectations.
Feasibility	Pass	A virtual queue can realistically be implemented within the prototype scope.
Testability	Need Improvement	Queue joining can be tested, but the expected observable outcome needs to be defined.

— The full analysis for all remaining initial requirements can be found in [Appendix A](#).

Improved Requirements

Following the quality analysis, requirements containing ambiguity, missing conditions, or weak testability were revised.

ID	Original Requirement	Revised Requirement	Reason for Change
FR-001	System shall allow user to join a virtual queue for a specific gym equipment	The system shall allow a logged-in member to join a virtual queue by selecting a piece of gym equipment and shall display the member's queue position within 2 seconds.	Defined the user, equipment selection, and observable result to improve clarity, completeness, and testability

— The full acceptance criteria for all functional requirements can be found in [Appendix C](#).

Acceptance Criteria

Acceptance criteria are defined for the key system behaviours using the Given-When-Then format.

Requirements	ID	Acceptance Criterion
FR-001	AC-01	Given a user is logged in and viewing an equipment list, when a user clicks "Join the queue" , then the user will automatically join the queue on that specific equipment and confirm their position within 2 seconds.

— The full acceptance criteria for all functional requirements can be found in [Appendix B](#).

Relevant Software Quality Attributes

Based on the requirements and identified quality risks, several software quality attributes are particularly relevant to GymQ. The attributes below were selected according to their importance to the queue management and equipment maintenance functions.

Quality Attribute	Relevance to GymQ
Functional Suitability / Correctness	Essential for ensuring queue positions, notifications, timeouts, fault reports, and equipment statuses behave according to the defined requirements.
Reliability	Important because queue and equipment information must remain consistent when multiple members interact with the system.
Performance Efficiency	Queue and equipment status changes need to be displayed promptly so members are not shown outdated information.
Usability	Gym members should be able to join queues and report equipment faults easily, while staff should be able to review reports without unnecessary complexity.
Security	Staff and administrative functions must be restricted to authorised users, particularly when confirming maintenance reports or managing equipment information.
Compatibility	The mobile interface is intended to operate across Android and iOS platforms.
Maintainability	The prototype should be structured so queue rules, equipment information, and maintenance functionality can be modified or extended in later development phases.

Proposed Solution and Initial Prototype

Proposed Solution Overview

Our solution to address the real-world problem is to build an app where users can digitally queue for gym machines and report faulty equipment in real time, replacing the current informal, verbal system.

Main Intended Functionality

Digital Queue

- Members join a virtual queue for a selected piece of equipment and see their queue position (FR-001)
- The next member in queue is notified automatically when equipment becomes available (FR-002)
- A next in queue member can send an in-app “nudge” to check whether the current user is still using the equipment with an automatic session end if there’s no response or the user confirms they’re finished (FR-003)
- Members who don’t claim their turn within 2 minutes of being noticed are automatically removed from the queue (FR-004)

Equipment Fault Reporting

- Members can submit a fault report for any piece of equipment with a description of the issue (FR-005)
- Staff review submitted reports and either confirm them as valid maintenance issues or reject them (FR-006)
- Equipment status automatically changes to “Unavailable” once a report is confirmed by staff (FR-007)

Equipment Usage Tracking

- The system records the start and end time of each member’s usage session, triggered by queue claim, manual finish, nudge outcome, or a 30-minute maximum duration cap, and stores the resulting duration for administrative reporting (FR-008)

Equipment Status Visibility

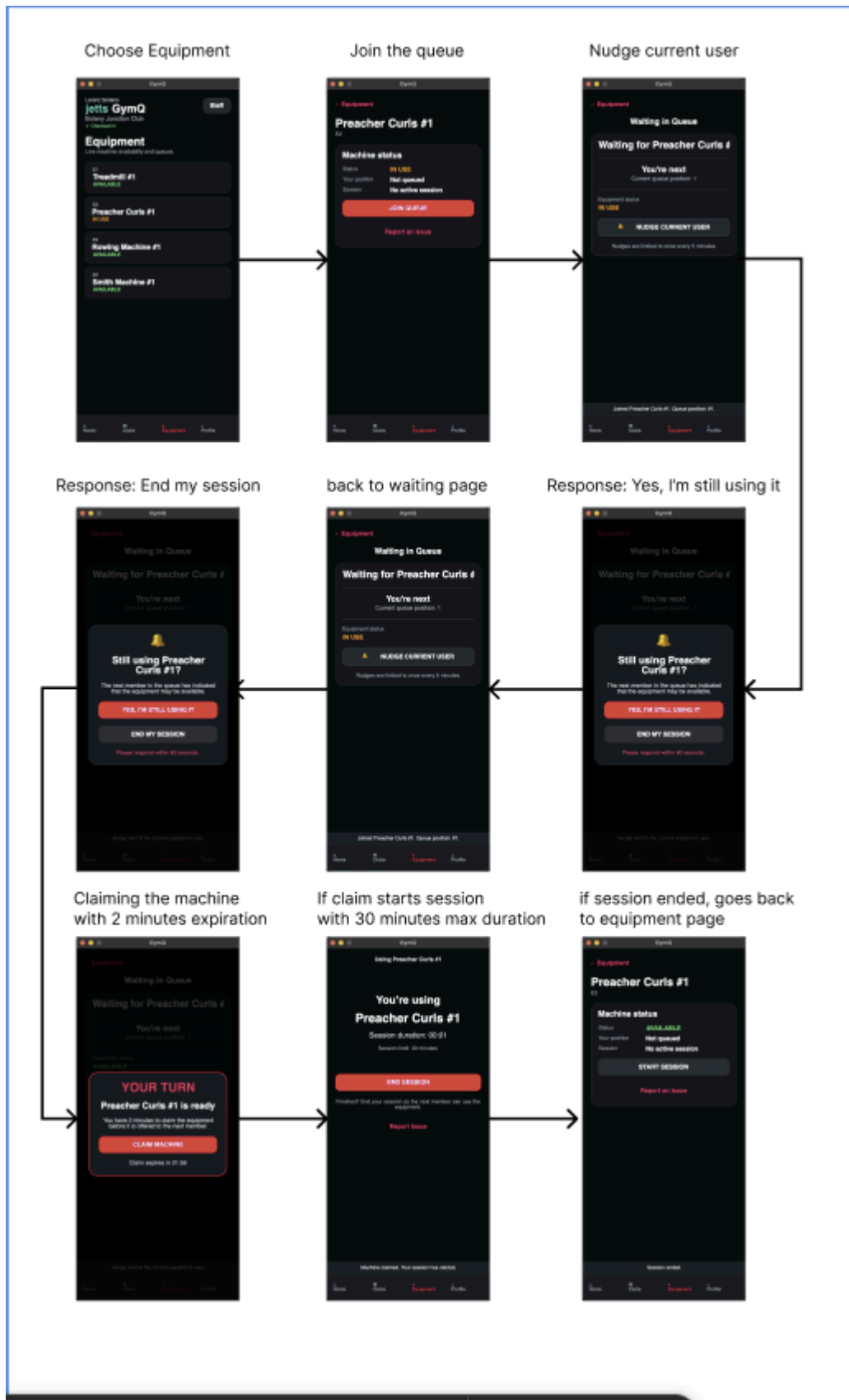
- Admins can view the current status of all gym equipment if it’s Available, In Use, or Unavailable at any time (FR-009)

How the Solution Addresses the Problem

Gyms currently rely on informal, verbal systems for equipment sharing and fault reporting, which doesn’t work during busy hours and means staff are unaware of issues until someone complains. GymQ solves this by letting members instantly check equipment status and join a queue remotely instead of physically waiting nearby, automatically notifying the next person when a machine frees up, and using timeouts to stop members from reserving equipment they never use. The nudge feature also helps when a machine looks “in use” but the

member actually isn't using it, letting the next person prompt them instead of confronting them in person. Lastly, fault reporting gives members a quick way to flag broken equipment, giving staff an actual record of issues instead of relying on word of mouth.

Initial Prototype: Joining Queue Workflow (GUI)



AI-Assisted Development Using GitHub Copilot

Where AI assistance was used

Module	What Copilot Supported	Team Member
FaultReportService	Thread-safety Improvements	Jayden
SessionService	Error-handling	Christian
QueueService	Design-stage review of FR-003 (nudge) implementation approach, including catching a cross-module double-notification risk before implementation	Lorenz

AI Supported Development

[Jayden]

Prompt:

“How can I improve my code in relation to quality assurance techniques”

GitHub Copilot output:

```
High-level QA recommendations

• Unit tests: add xUnit tests for happy paths and edge cases (nulls, unauthorized staff, already-reviewed reports, non-existent equipment). Use mocking (Moq/NSubstitute) for IEquipmentRepository.
• Integration tests: cover repository persistence and equipment-status side effects.
• Static analysis & formatting: enable analyzers, run dotnet format in CI. I prepared a recommended .editorconfig and CONTRIBUTING.md (see note below) to enforce style + QA rules.
• CI pipeline: require build, tests, analyzers, and coverage upload on PR. Gate merges on green CI.
• Code review checklist: tests present, edge cases handled, logging present, no thread-safety issues, no breaking public API unless intentional.
• Observability: add logging (ILogger) for key events and failures to assist debugging and test failures.
• Thread-safety & determinism: avoid non-thread-safe collections and non-deterministic ID generation in services used concurrently.
• Separation of concerns: keep business logic in services, repo operations in repository interfaces (consider adding update methods to repository rather than mutating returned objects if persistence requires it).
• Error handling: use clear exception types and messages; consider a domain-specific exception for expected business errors if useful.
• Coverage target: set a baseline (e.g., 80%) for new/changed code and enforce in CI.

Concrete refactor for FaultReportService.cs

• Make in-memory storage thread-safe (ConcurrentDictionary).
• Make ID generation atomic using Interlocked.Increment.
• Add optional ILogger<FaultReportService> dependency.
• Fix missing using System.Linq; and add required usings.
• Keep the public surface compatible while improving reliability and testability.
```

After reviewing the AI suggestions for how we can improve our quality assurance, some relevant suggestions that could be implemented within the scope of an initial prototype are:

- Make the in-memory equipment storage thread-safe by switching to a ConcurrentDictionary instead of a List type.
- Make ID generation atomic using Interlocked.Increment, instead of a simple integer that increments by 1 each time.

Implementing selected suggestions:

```
public class FaultReportService
{
    // Thread-safe in-memory store for the prototype.  ### AI suggested improvement
    private readonly ConcurrentDictionary<string, FaultReport> _reports = new();

    // Atomic counter for generating unique report IDs (instead of a simple integer) ### AI suggested improvement
    private long _nextReportId = 0;
}
```

_reports has changed from a List to a ConcurrentDictionary for thread safety, with some syntax and logic changes to support the switch.

```
var report = new FaultReport
{
    ReportId = $"R-{Interlocked.Increment(ref _nextReportId)}", //### AI suggested improvement
    EquipmentId = equipmentId,
    SubmittedByMemberId = member.MemberId,
    Description = description,
    Status = FaultReportStatus.Pending,
    SubmittedAt = DateTime.UtcNow
};
```

The incrementation logic has changed from a simple _nextReportId++ to an Interlocked Increment for atomic ID generation.

[Christian]

Prompt:

“Act as a .NET software engineer doing a code review for a software quality assurance course. Review [SessionService.cs](#). State and suggest improvements in [SessionService.cs](#)”

Github copilot output:

Senior Code Review: SessionService.cs

State of the Code & Improvement Recommendations

Current State Assessment

SessionService is functionally sound for a prototype but requires improvements in:

- **Error handling consistency** (silent failures vs. exceptions)
- **API completeness** (missing bulk session queries for admin reporting)
- **Production readiness** (no enforcement mechanism for 30-minute cap)
- **Code quality** (typos, comments, null-safety)
- **Encapsulation** (mutable state split between UsageSession and external callers)

Below are specific improvements organized by category.

Based on the assessment, Copilot flagged two issues we evaluated closely: the silent return in EndSession when no active session exists, and the use of KeyNotFoundException rather than ArgumentException for an unrecognised equipmentId. Both suggestions were accepted and implemented. Also upon reviewing all suggested improvements I ended up not accepting it as it needs proper evaluation across requirements.

[1] EndSession silent return:

I accepted this because a caller invoking EndSession on equipment with no active session likely reflects a genuine logic error upstream, and surfacing it is more useful than failing silently. This change required modifications within the initial test cases for the ServiceSession module which led to 2 failing tests, asserting a wrong exception. Both was fixed.

Before Code:

```
// Checks if the session does not exists
    if(session == null)
    {
        return;
    }
```

Improved AI suggested code:

```
// Checks if the session does not exists
    if(session == null)
    {
        throw new InvalidOperationException(
            $"No active session found for equipment '{equipmentId}'. " +
            "Session may have already been ended or equipment was never
in use."
        );
    }
```

[2] Exception type for unknown equipment:

We accepted this since ArgumentException more accurately reflects that the failure originates from invalid caller supplied value not a system-level lookup failure. Test cases were changed accordingly.

Before code:

```
// Finds an equipment via equipmentId through _equipment if not throws exception
    if (!_equipment.TryGetValue(equipmentId, out var equipment))
    {
        throw new KeyNotFoundException($"Equipment '{equipmentId}' not
found in the system.");
    }
```

Improved AI suggested code:

```
if (!_equipment.TryGetValue(equipmentId, out var equipment))
{
    throw new ArgumentException(
        $"Equipment '{equipmentId}' not found in the system.",
        nameof(equipmentId));
}
```

[Lorenz]

Prompt used

```
← Queue management implementation approach 3/4

I am implementing Person A's Queue Management module for the GymQ C# prototype.

I am implementing Person A's Queue Management module for the GymQ C# prototype.

Please review the existing QueueService.cs, Member.cs, SessionService.cs, and QueueServiceTests.cs files before suggesting changes.

I am now working on FR-003: Nudge Handling, specifically:

• SendNudge(string equipmentId, string fromMemberId)
• HandleNudgeResponse(string equipmentId, bool stillUsing)

Current intended behaviour:

SendNudge

• Only the member who is currently next in the queue for that equipment should be allowed to send a nudge.
• A cooldown of 1 nudge per equipment every 5 minutes must be enforced.
• If the cooldown has not expired, the method should return false.
• If the nudge is allowed, the method should return true.
• The actual notification mechanism is not implemented yet because this phase is business-logic only and has no GUI.
• A 1-minute response window is mentioned in the requirement, but there is currently no background timer/scheduler implementation.

HandleNudgeResponse

• If stillUsing == true, the current session should continue and no further action should occur.
• If stillUsing == false, this represents either an explicit "Finished" response or a timeout.
• In that case, the current session should eventually end through Person C's SessionService.EndSession(...).
• After the session ends, NotifyNextInQueue(equipmentId) should be triggered.
• Do not unnecessarily modify Person C's SessionService or shared models.
• Do not change the existing public method signatures unless absolutely necessary and explain why first.
```

Please provide:

1. A recommended implementation approach for both SendNudge() and HandleNudgeResponse().
 2. Any additional private state or helper fields that may be needed, such as tracking the last nudge time per equipment.
 3. Design concerns or ambiguous requirements I should discuss with the team before implementing.
 4. Appropriate MSTest unit test cases covering positive, negative, and edge cases.
 5. Descriptive test names using the format Method_Scenario_ExpectedResult.
 6. Advice on how to test the 5-minute cooldown without introducing slow real-time waits in unit tests.
 7. Advice on how to handle the dependency on SessionService.EndSession() while Person C's module is being developed separately.
 8. Identify which tests can be implemented now and which should wait until integration testing.
- Do not automatically modify files. Explain the approach first so I can review and understand the recommendations before implementing anything.

Github copilot output – key finding

Important Design Decisions

1. Duplicate notification ownership

The skeleton says SessionService.EndSession() should call QueueService.NotifyNextInQueue(), while HandleNudgeResponse() also says it should call QueueService.NotifyNextInQueue() after ending the session. Implementing both literally may notify the next member twice.

Choose one owner:

- SessionService.EndSession() ends the session and advances the queue, or
- QueueService.HandleNudgeResponse() ends the session and advances the queue.

The cleaner option is usually for EndSession() to publish an event or invoke a callback, with one component responsible for queue advancement.

While reviewing the intended behaviour for SendNudge() and HandleNudgeResponse(), Copilot flagged a design conflict before any code was written: the module skeleton's comments indicated that SessionService.EndSession() should call

`QueueService.NotifyNextInQueue()`, while `HandleNudgeResponse()` was also expected to call `NotifyNextInQueue()` after ending the session. Implemented literally, both paths firing would notify the next queued member twice for a single session-end event.

Copilot proposed two ownership options rather than picking one for us:

- `SessionService.EndSession()` owns ending the session and advancing the queue, or
- `QueueService.HandleNudgeResponse()` owns both.

Our decision:

```
[TestMethod]
0 references
public void HandleNudgeResponse_Finished_EndsSessionAndMakesEquipmentAvailable()
{
    var equipment = new Equipment("SquatRack2", "Squat Rack #2");

    var equipmentStore = new Dictionary<string, Equipment>
    {
        [equipment.EquipmentId] = equipment
    };

    var sessionService = new SessionService(equipmentStore);

    sessionService.StartSession("SquatRack2", "M001");

    var queueService = new QueueService(sessionService);

    queueService.JoinQueue(
        "SquatRack2",
        new Member("M002", "Mia"));

    queueService.HandleNudgeResponse(
        "SquatRack2",
        false);

    Assert.AreEqual(
        EquipmentStatus.Available,
        equipment.Status);
}
```

We chose the second option. `HandleNudgeResponse()` calls `_sessionService.EndSession(equipmentId, SessionEndReason.NudgeResponse)` and then `NotifyNextInQueue(equipmentId)` itself; `SessionService.EndSession()` was kept independent of queue-advancement logic, so it doesn't call `NotifyNextInQueue()` on its own. This keeps a single, unambiguous owner for the "session ended → next member notified" step, verified in `HandleNudgeResponse_Finished_EndsSessionAndMakesEquipmentAvailable()` and the cross-module integration tests.

Why this matters for Task 4:

This is a case where Copilot's contribution wasn't code we typed in verbatim — it was catching a coordination bug at the design stage, between two modules being built by different team members in parallel, before it became a runtime double-notification defect. [Teammate]'s later review independently flagged the same risk, which we treat as validation that the design concern was real rather than an AI false positive.

A suggestion we evaluated and rejected:

A good design is a private clock delegate with an internal test constructor:

```
private readonly Func<DateTime> _utcNow;

public QueueService()
    : this(() => DateTime.UtcNow)
{
}

internal QueueService(Func<DateTime> utcNow)
{
    _utcNow = utcNow;
}
```

The test can then control time:

```
var currentTime = new DateTime(2026, 8, 27, 10, 0, 0, DateTimeKind.Utc);
var service = new QueueService(() => currentTime);

service.JoinQueue("SquatRack2", new Member("M001", "Enzo"));

Assert.IsTrue(service.SendNudge("SquatRack2", "M001"));

currentTime = currentTime.AddMinutes(5);

Assert.IsTrue(service.SendNudge("SquatRack2", "M001"));
```

Copilot also recommended injecting a `Func<DateTime>` clock delegate into `QueueService` so the 5-minute nudge cooldown could be tested deterministically without real-time waits. We decided not to adopt this for the current prototype phase — it would have required an additional internal constructor and `InternalsVisibleTo` wiring purely for test purposes, adding complexity we judged wasn't worth it yet given `DateTime.UtcNow` is only exercised in a small number of cooldown tests. This is a candidate for revisiting if concurrency/time-based test coverage expands in Phase 2.

Initial Test Strategy and Test Planning

Testing Objectives

Testing at this stage is meant to test that the core business logic behind queueing, nudging, claim timeouts, fault reporting, and session tracking actually fits the functional requirements and acceptance criteria, not just that each method runs without errors. A method can run without errors and still return the wrong queue position or skip a state change, so test cases are written to fit the expected behaviour for the acceptance criteria.

In-Scope Testing

Queue logic (FR-001–004), fault reporting (FR-005–007), session tracking (FR-008–009), cross-module interactions, and, since the UI is being built, user flows through the actual interface (joining a queue, submitting a fault report).

Out-of-Scope Testing

Accessibility testing, cross-device/browser compatibility, real push notification delivery, and deployment to actual gym hardware/networks remain out of scope.

Test Levels

Unit testing — implemented using MSTest, with a substantial set of test methods covering `QueueService`, `SessionService`, and `FaultReportService`, continuing as modules evolve.

Integration testing — cross-module flows have already started, e.g. `HandleNudgeResponse` triggering `SessionService.EndSession()` is covered by an existing test. This verifies the complete session-end → queue-advance workflow, i.e. confirming that once a session ends, the queue correctly advances to notify the next member, without assuming any single service owns the notification step itself. Dedicated cross-module integration tests now also cover the full claim → session-start flow and the fault-confirmation → session-block flow.

System testing — since the team is actively building the UI within this same project cycle, system-level testing of full user flows through the actual interface (joining a queue, submitting a fault report, reviewing a report as staff) is planned work for this same phase, not a future placeholder.

Regression testing — rerunning the full test suite whenever a module changes, since `QueueService`, `FaultReportService`, and `SessionService` all share the `Equipment` and `Member` models. The suite is currently run after every merge to catch regressions early.

Functional Testing Approach

Test cases follow the Given-When-Then acceptance criteria from Task 2, implemented as MSTest methods for each requirement.

Non-Functional Testing Approach

Focused mainly on concurrency/reliability, since SessionService already uses locking around session start/end, this should be verified with simulated concurrent calls. For performance (NFR-001), execution time can be measured now (e.g. how long JoinQueue() or NotifyNextInQueue() takes to run), but the full verification of the 2-second requirement, which includes the UI and notification delivery, will only be useful once that is integrated.

Entry Criteria:	Exit Criteria:
• The module under tests builds clean. Dotnet build zero errors • Any change to a shared model (Equipment, Member, EquipmentStatus, QueueEntry) is agreed with the other module owner first. • Fakes for shared dependencies (IEquipmentRepository, TimeProvider) exist before dependent tests are written against them.	• Dotnet test report failing tests across all four classes. • Every in scope FR has at least one passing test asserting its behaviour. • No new compiler warnings introduced by the change under review. • Any failed tests are immediately fixed, and properly documented.

Test Environment

Local development using .NET/C#, built and run with Visual Studio or the dotnet CLI. The prototype UI is being built using Avalonia, so manual interface testing can be carried out on both Windows and macOS when UI screens are completed. The data and storage remains in-memory for the prototype, with no external database or server dependency.

Test Tools

MSTest for unit and integration testing, and GitHub for version control, tracking changes to test files over time, and reviewing test-related commits as part of the team's contribution history.

Initial Test Cases and Requirements Traceability

Initial Test Cases

Test Case	Requirements	Description	Result
JoinQueue_FirstMember_ReturnsPositionOne()	FR-001	Logged-in member joins virtual queue; position displayed within 2s	Passed
JoinQueue_EmptyEquipmentId_ThrowsArgumentException()	FR-001	Join request with empty equipment ID is rejected	Passed
NotifyNextInQueue_QueueHasMultipleMembers_NotifiesOnlyFrontMember()	FR-002	Next member in queue notified when equipment becomes available; others not notified	Passed
SendNudge_MembersNotNextInQueue_ReturnsFalse()	FR-003	Only the next-in-queue member is permitted to send a nudge	Passed
HandleNudgeResponse_Finished_EndsSessionAndMakesEquipmentAvailable()	FR-003, FR-008	Nudge response ("finished") ends session and frees equipment	Passed
EnforceClaimTimeout_TimeoutFrontMember_RemovesMemberAndNotifiesNext()	FR-004	Member removed from queue after 2-minute claim timeout; next member notified	Passed
SubmitFaultReport_ValidInput_ReturnsPendingReport()	FR-005	Member submits a fault report with valid description; status set to Pending	Passed
ReviewFaultReport_NonStaffMember_ThrowsUnauthorizedAccessException()	FR-006, NFR-003	Non-staff member is blocked from reviewing/confirming a fault report	Passed
ReviewFaultReport_Confirmed_SetsEquipmentStatusUnavailable()	FR-007	Equipment status changes to Unavailable when report is confirmed by staff	Passed

StartSession_ValidEquipment_ReturnsActiveSessionAndSetsEquipmentInUse()	FR-008	Session starts correctly and sets equipment status to In Use	Passed
GetSessionDuration_EndedSession_ReturnsElapsedTime()	FR-008	Session duration correctly calculated and stored after session ends	Passed
EnforceMaxSessionDuration_SessionReached30Minutes_EndsSessionWithMaxDurationReason()	FR-008	Session automatically ends after 30-minute maximum duration is reached	Passed
GetAllEquipmentStatus_ReturnsEveryEquipmentWithCurrentStatus()	FR-009	Admin can view current status of all gym equipment	Passed
ClaimEquipment_NotifiedFrontMember_StartsSessionAndUpdatesQueueAndEquipment()	FR-001, FR-002, FR-008, NFR-001	Full claim flow across modules: queue, session, and equipment status update together	Passed
ConfirmedFaultReport_MarksEquipmentUnavailable_AndBlocksNewSession()	FR-006, FR-007, FR-008, NFR-002, NFR-006	Confirmed fault report marks equipment unavailable and prevents a new session from starting	Passed

Initial requirements traceability matrix

Requirement ID	Requirement Summary	Test Case(s)
FR-001	Logged-in member joins virtual queue; position displayed within 2s	JoinQueue_FirstMember_ReturnsPositionOne(), JoinQueue_EmptyEquipmentId_ThrowsArgumentException(), ClaimEquipment_NotifiedFrontMember_StartsSessionAndUpdatesQueueAndEquipment()
FR-002	Next member in queue	NotifyNextInQueue_Queue

	notified when equipment becomes available	HasMultipleMembers_Notifies_OnlyFrontMember(), ClaimEquipment_NotifiedFrontMember_StartsSessionAndUpdatesQueueAndEquipment()
FR-003	Next-in-queue member can nudge current user; auto-end on no response/finish	SendNudge_MemberIsNotNextInQueue_ReturnsFalse(), HandleNudgeResponse_Finished_EndsSessionAndMakesEquipmentAvailable()
FR-004	Member removed from queue if not claimed within 2 minutes	EnforceClaimTimeout_TimeoutFrontMember_RemovesMemberAndNotifiesNext()
FR-005	Member submits fault report with description	SubmitFaultReport_ValidInput_ReturnsPendingReport()
FR-006	Staff reviews report — confirm as maintenance, or reject	ReviewFaultReport_NonStaffMember_ThrowsUnauthorizedAccessException(), ConfirmedFaultReport_MarkEquipmentUnavailable_AndBlocksNewSession()
FR-007	Equipment status set to "Unavailable" on confirmed report	ReviewFaultReport_Confirmed_SetsEquipmentStatusUnavailable(), ConfirmedFaultReport_MarkEquipmentUnavailable_AndBlocksNewSession()
FR-008	Session start/end time and duration recorded across all end triggers	HandleNudgeResponse_Finished_EndsSessionAndMakesEquipmentAvailable(), StartSession_ValidEquipment_ReturnsActiveSessionAndSetsEquipmentInUse(), GetSessionDuration_EndedSession_ReturnsElapsedTime(), EnforceMaxSessionDuration_SessionReached30Minutes_EndsSessionWithMaxDurationReason(), ClaimEquipment_NotifiedFrontMember_StartsSessionAndUpdatesQueueAndEquipment(), ConfirmedFaultReport_MarkEquipmentUnavailable_AndBlocksNewSession()

FR-009	Admin views current status of all gym equipment	GetAllEquipmentStatus_ReturnsEveryEquipmentWithCurrentStatus()
NFR-001	Queue status updates within 2 seconds under normal load (Performance)	ClaimEquipment_NotifiedFrontMember_StartsSessionAndUpdatesQueueAndEquipment()
NFR-002	Usage/maintenance logs retained ≥ 6 months (Reliability)	ConfirmedFaultReport_MarksEquipmentUnavailable_AndBlocksNewSession() (partial — confirms correct state persistence, not retention duration itself)
NFR-003	Only authorized staff accounts can submit maintenance reports (Security)	ReviewFaultReport_NonStaffMember_ThrowsUnauthorizedAccessException()
NFR-004	System usable by first-time users without training (Usability)	Not yet covered — requires manual/usability testing, planned for a later phase
NFR-005	Mobile interface works on Android and iOS (Compatibility)	Not yet covered — requires device testing, planned for a later phase
NFR-006	System available 99% of the time during operating hours (Reliability)	EnforceMaxSessionDuration_SessionReached30Minutes_EndsSessionWithMaxDurationReason(), ConfirmedFaultReport_MarksEquipmentUnavailable_AndBlocksNewSession() (partial — confirms graceful state handling, not uptime measurement itself)

A requirements traceability matrix serves two connected purposes in an SQA project like GymQ: verifying that quality work is complete, and controlling the impact of change.

For quality assurance, the matrix provides direct, checkable evidence that every stated requirement has actually been verified by at least one test case, closing the loop between what was specified (Task 2) and what was tested (Task 6). This matters because it's easy, without traceability, for a requirement to quietly go untested, or for a test suite to grow tests that don't map back to any real stakeholder need.

For change management, the matrix makes the impact of any future change immediately visible without needing to re-read or re-reason through the whole codebase. If FR-003 needed to change; say, that the cooldown move from 5 minutes to 10 minutes the matrix shows exactly which test cases need to be re-reviewed or re-run rather than requiring a full re-test of the entire system.

Project Progress, Risks, and Next Steps

Summary of work completed so far

- All three core modules implemented as independent, testable service classes
 - QueueService
 - FaultReportService
 - SessionService
- Built around shared Equipment and Member modes, which satisfies FR-001 through FR-009
- 72 MSTest methods written across four test classes:
 - QueueServiceTests - 35 tests which involves, queue join, position tracking, notification, nudges, claim timeout
 - FaultReportServiceTests - 15 tests which involves, fault submission, staff review, and status cascade
 - SessionServiceTests - 18 tests which involve session start/end, duration tracking, 30-minute cap enforcement.
 - CrossModuleIntegrationTests - 4 tests verifies cross-module flows including queue claim to session start to session end to queue advances to confirmed fault report to equipment unavailable to new session blocked
- Logic-first development approach followed as planned - business rules, state transitions, and edge cases. Built and unit-tested before any UI work began.
- Basic Avalonia application shell scaffolded - implement a basic user interface, but not fully.
- Foundation in place for Task 6 traceability where every functional requirement now has real, executable test coverage rather than only planned test cases.

Technical Limitations

- In-memory data storage only
- Not fully implemented GUI
- No authentication system
- No physical hardware or sensors

Testing Gaps

- Both NFR-004 (usability) and NFR-005(compatibility) are not yet covered
- Test coverage for concurrency/race conditions is limited

Integration Risks

- Cross-module coordination
- Increases the risk of merge conflicts across three people working in parallel modules

Scope Risks

- Nudge misuse mitigation is basic
- Limited semester time

How Risks Will Be Managed in the Next Phase

- A concurrency review will be done for all shared in-memory collections using simulated concurrent access tests to confirm thread safety.
- GUI-to-backend integration needs to be prioritised early in the next phase so that acceptance that involves observable behaviour (eg. time-based behaviour) can be tested with a real interface rather than just backend logic.
- Any AI-assisted code changes from GitHub Copilot will continue to be reviewed before being integrated or using the suggestions, rather than being accepted because it “looks correct”.
- Regular use of GitHub for version control needs to continue throughout the rest of the project.

Plan for Completing the Final Project

1. Complete the remaining logic for fault reporting and session tracking modules, and ensure all acceptance criteria have an implemented behaviour.
2. Polish the GUI screens and implement a more in-depth GUI, rather than a basic GUI with no formatting and no theme.
3. Expand the MSTests to fully cover all the acceptance criteria, including concurrency tests for queue and session logic with simulated simultaneous access.
4. Ensure the GUI and backend are fully integrated, checking that the exit criteria (no unfinished placeholders, stable shared models, all tests passing, no known critical errors) are met.

Team Reflection on AI assisted development

Using Github Copilot across three modules gave the team a faster starting point for repetitive scaffolding and, in Lorenz’s case, surfaced a genuine cross-module design risk. A potential

double notification bug between `SessionService.EndSession()` and `QueueService.HandleNudgeResponse()`. Before any code was written to implement it. That the risk was independently confirmed by a teammate's review afterward gave the team confidence it was a real issue rather than an AI false positive, which shaped how we now treat Copilot's suggestion as a second opinion worth checking, not an instruction to follow automatically.

This was reinforced by the two suggestions evaluated for [SessionService.cs](#), that we chose not to apply, after considering how they'd interact with behaviour our own tests already depended on. Working across three parallel modules also meant AI-assisted suggestions sometimes needed to be discussed as a team before being accepted, since a change proposed for a one module directly affected another team member's code. Overall, copilot supported productivity and helped surface design questions earlier than they might otherwise have come up, but every suggestion still required the same scrutiny as any other code change before merging.

Appendices:

Appendix A — Full list of initial requirements analysis.

FR-001- The system shall allow users to join a virtual queue for a specific gym equipment.

Criterion	Assessment	Finding
Clarity	Need Improvement	The main function is understandable, but the type of user and meaning of a selected piece of equipment should be clearly defined.
Completeness	Need Improvement	Conditions for joining and the expected result after joining are not specified.
Consistency	Review Required	Must be checked against related queue requirements such as FR-002 and FR-003
Correctness	Verification Required	The function supports the proposed queueing problem but should be confirmed against stakeholder expectations.
Feasibility	Pass	A virtual queue can realistically be implemented within the prototype scope.
Testability	Need Improvement	Queue joining can be tested, but the expected observable outcome needs to be defined.

FR-002 - The system shall notify the next user in queue when equipment becomes available

Criterion	Assessment	Finding
Clarity	Need Improvement	The word “notify” is ambiguous. The notification method isn’t stated like is it Push notification, on-screen

		alert or SMS?
Completeness	Issue	What information should the notification contain? What happens after they are notified?
Consistency	Pass	There are no conflict with other requirements, as we define other queue requirements consistently as "next in queue"
Correctness	Verification Required	The intent is correct but still requires confirmation whether the notification is actually what the stakeholders need or a live-updating display.
Feasibility	Pass	Realistic within a semester project, as we only going to implement an in-app/on-screen notification
Testability	Issue	It's not fully testable as the word "Notify" has no measurable success condition.

FR-003 - The system shall allow a member who is next in queue to send an in-app notification to the current equipment user, asking them to confirm whether they are still using the equipment.

Criterion	Assessment	Finding
Clarity	Need Improvement	Needed specification in what happens after the nudge is sent.
Completeness	Issue	The response window is missing, what happens if there's no response, and any cooldown/rate-limiting to prevent nudge spam.
Consistency	Pass	It fits perfectly alongside the requirements of session-tracking and queue-notification requirements without conflict.

Correctness	Pass	It reflects the gap we identified from FR-007. As we are only based on user input this is a legitimate stakeholder need.
Feasibility	Pass	Simple to implement as it does not need a new infrastructure.
Testability	Issue	No defined timeout or resulting system state.

FR-004 - System shall remove a user from queue if they don't claim equipment within a set of time

Criterion	Assessment	Finding
Clarity	Need Improvement	"Within a set of time" is ambiguous. It is not set how long the system will have to wait before it removes users from the queue.
Completeness	Issue	It does not specify what happens after a user is removed from the queue.
Consistency	Verification Required	It should align with the notification requirement as FR-002 does not specify a specific timeframe.
Correctness	Pass	It reflects the real and necessary behaviour. Without this, a non-responsive member could block the queue indefinitely, which is a legitimate problem to solve.
Feasibility	Pass	Realistic to implement within the scope. It is a straightforward timeout/state-transition rule.
Testability	Issue	It is not testable as written. There is no specified fixed time value, no defined trigger point, no defined end state, and there's no way to write a concrete pass/fail

		test.
--	--	-------

FR-005 - The system shall allow a gym member to flag/report fault equipment.

Criterion	Assessment	Finding
Clarity	Issue	The term “flag/report” is ambiguous because it is unclear where flagging and report are the same action. It also does not specify how the member identifies the faulty equipment or what information must be provided.
Completeness	Issue	The requirement does not specify what information the member must provide when reporting a fault or what happens after the report is submitted.
Consistency	Pass /Verification Required	It is consistent with the intended maintenance workflow provided FR-005 handles staff review of the submitted fault report.
Correctness	Pass	Allowing members to report faulty equipment supports the identified problem of improving communication about equipment faults.
Feasibility	Pass	A basic fault-reporting feature is realistic for the prototype and can be implemented using a simple form.
Testability	Issue	The requirement does not define the required report information or a clear expected outcome, making it difficult to establish a concrete pass/fail test.

FR-006 - Staff shall be able to submit a maintenance report for malfunctioning equipment

Criterion	Assessment	Finding
Clarity	Issue	Submitting a maintenance report is so vague, that it doesn't know what the maintenance report must contain. Also the "malfunctioning" is too broad where it does not define what type of issues the equipment has or the severity of the equipment state.
Completeness	Issue	It only specifies staff who can only submit the report, but members can also flag faulty equipment. It also does not state what happens after submission, does this straight notify maintenance technicians? Or does the equipment status change straight away to unavailable.
Consistency	Verification Required	It should align with the fault status lifecycle such as report - triage - resolve - close. It can conflict with other requirements that assume equipment becomes unavailable once reported.
Correctness	Pass	Reflects genuine stakeholder needs. Staff need a way to formally log faults rather than relying on verbal communication, which is the core problem we're trying to solve.
Feasibility	Pass	It is a straightforward functional requirement, realistic within scope.
Testability	Issue	There are no defined field, no defined system response after submission, there is no way to write a concrete pass/fail test case

FR-007 - System shall mark equipment as “unavailable” once a maintenance report is filed

Criterion	Assessment	Finding
Clarity	Need Improvement	It is fairly clear, but one thing is not specified is what happens afterward once a maintenance report is filed. Does this automatically remove users from the queue, or does this mean the specific equipment will have a greyed out display?
Completeness	Issue	It does not specify what happened to members who are already on the queue, or does the queue still be the same order once the equipment is fixed or will be reset?
Consistency	Pass	It is consistent with the FR-004, which already stated status faulty/unavailable once the report is filed.
Correctness	Pass	Reflects the correct real-world behaviour, equipment shouldn't be queueable while broken.
Feasibility	Pass	Simple state change logic, which is realistic within scope.
Testability	Issue	Needs a precise, observable system state such as status field, visual indicator, or absence from queue list to be verifiable as pass/fail.

FR-008 - System shall track equipment usage duration per session

Criterion	Assessment	Finding
Clarity	Issue	“Equipment usage duration per session” is ambiguous in terms of whether the system tracks the duration of

		members while using the equipment or the overall usage of the equipment like in a week, or like in a month.
Completeness	Issue	It does not state what is the purpose of this requirement if it's for maintenance or for usage analytics. The requirement is pretty vague in terms of what it is for? Who is it for?
Consistency	Review Required	It depends on the clarity of what this requirement is for? Is it for members on how long they use one equipment per session. Or is it for usage analytics for maintenance scheduling?
Correctness	Verification Required	It needs verification on what it is for? Because it can help strengthen the queue system where sessions vary depending on busy periods. and also can be beneficial for fault reporting.
Feasibility	Pass	It is feasible, as we only implement a simple time start/ end log. Still depends on where it is for and how big the scope is for this requirement.
Testability	Issue	Not testable as written, as there are no trigger points, or no defined unit measurement.

FR-009 - Admin shall be able to view real-time status of all gym equipments

Criterion	Assessment	Finding
Clarity	Issue	"Real-time" is vague. It does not specify what real-time means is it to display the current states of equipment ? Live-update of all equipment? Does it show what's the status of every equipment either available

		or in-use, and etc.
Completeness	Issue	It does not specify what it shows? Real-time status can be a lot of things such as equipment status? Who is in the queue? Also it does not specify where this view lives, is it for members or just admins?
Consistency	Pass	It aligns with the other requirements.
Correctness	Pass	It is a clear stakeholder need. Admin/staff must have access with what the equipment's status, what's being flagged as faulty, or what's not available to use.
Feasibility	Need Improvement	It is feasible, depending on what "Real-time" really means. It depends on what we decide on the scope of this real-time update.
Testability	Issue	There are no defined "real-time" latency threshold or defined unit measurement to test to.

Appendix B — Full list of acceptance criteria

Requirements	ID	Acceptance Criterion
FR-001	AC-01	Given a user is logged in and viewing an equipment list, when a user clicks “Join the queue” , then the user will automatically join the queue on that specific equipment and confirm their position within 2 seconds.
FR-002	AC-02	Given a user is already next on queue in an equipment store, when the equipment’s status becomes “available”, then an in-app notification should pop-up and notify the next user that it is their turn.
FR-003	AC-03	Given a user has still an ongoing session but no one is using the equipment anymore, when the next in queue user ‘nudge’ the current user, then an in-app notification will notify the current user of the equipment and needed to respond whether the user still uses the equipment or not, if no respond within 1 minute their session will end automatically and the next in queue can claim the position.
FR-004	AC-04	Given a user has been notified that the selected equipment is available, when the user does not claim the equipment within 2 minutes of receiving the notification, then the system shall automatically remove the user from the virtual queue.
FR-005	AC-05	Given a gym member has selected a piece of gym equipment, when the

		member provides a description of the issue and submits the fault report, then the system shall record the fault report and make it available for staff review.
FR-006	AC-06	Given a fault report has been submitted by a gym member, when staff review the report, then staff shall be able to either confirm it as a formal maintenance report or reject it.
FR-007	AC-07	Given a submitted fault report has been reviewed by staff, when staff confirm it as a formal maintenance report, then the system shall change the selected equipment's status to “Unavailable.”
FR-008	AC-08	Given a user begins an equipment-use session, when the user ends the session, then the system shall record the session's start time, end time, and total usage duration.
FR-009	AC-09	Given an administrator is logged into the system, when the administrator views the equipment status page, then the system shall display the current status of all gym equipment, including “Available,” “In Use,” or “Unavailable.”

Appendix C - Full list of improved requirements

ID	Original Requirement	Revised Requirement	Reason for Change
FR-001	System shall allow user to join a virtual queue for a specific gym equipment	The system shall allow a logged-in member to join a virtual queue by selecting a piece of gym equipment and shall display the member's queue position within 2 seconds.	Defined the user, equipment selection, and observable result to improve clarity, completeness, and testability
FR-002	System shall notify the next user in queue when a gym equipment becomes available	The system shall display an in-app notification to the next user in the virtual queue when the selected gym equipment becomes available.	Clarified the notification method and the intended recipient.
FR-003	The system shall allow a member who is next in queue to send an in-app 'nudge' notification to the current equipment user, asking them to confirm whether they are still using the equipment.	The system shall allow a member who is next in queue to send an in-app "nudge" notification to the current equipment user, asking them to confirm whether they are still using the equipment. If the current user does not respond within 1 minute, or responds that they are finished, the system shall automatically end their session and notify the next queue member that the equipment is available. To prevent misuse, a member shall be limited to sending one nudge per equipment item every 5 minutes.	It needs clarity such as defined triggers, response window, and the outcome. It is not complete in terms of response and no-response cases, and also abuse mitigation. And needs unit measurements for testability.

FR-004	System shall remove a user from queue if they don't claim equipment within a set time	The system shall remove a user from the virtual queue if they do not claim the available equipment within 2 minutes of being notified.	Replaced the vague "set time" with the 2-minute limit to improve clarity and testability.
FR-005	The system shall allow a gym member to flag/report fault equipment.	The system shall allow a gym member to submit a fault report for a selected piece of gym equipment by providing a description of the issue.	Clarified the reporting action, equipment selection, and required fault information to improve clarity, completeness, and testability.
FR-006	Staff shall be able to submit a maintenance report for malfunctioning equipment	The system shall allow staff to review a submitted report and confirm it as a formal maintenance report or reject the report.	Defined the staff review process and possible outcomes to distinguish a member-submitted fault report from a staff-confirmed maintenance report.
FR-007	System shall mark equipment as "unavailable" once a maintenance report is filed	The system shall change the equipment status to "Unavailable" when its fault report is confirmed as a maintenance report by staff.	Defined the exact trigger and resulting equipment state to improve clarity, consistency, and testability.
FR-008	System shall track equipment usage duration per session	The system shall record the start time of a member's equipment usage session when they claim their turn from the queue, and shall record the end time when the session is ended, whether by the member manually marking themselves finished, by a successful nudge response or nudge timeout, or by the system-enforced	Defined what constitutes usage duration and the information recorded for each session, improving clarity and testability.

		maximum session duration of 30 minutes. And shall store the resulting session duration for administrative reporting.	
FR-009	Admin shall be able to view real-time status of all gym equipments	The system shall allow an admin to view the current status of all gym equipment, including "Available," "In Use," and "Unavailable."	Replaced the ambiguous term "real-time" with clearly defined equipment states, improving clarity, completeness, and testability.